

Objetivo

Comprobar la separación lógica de recursos con nombres equivalentes en namespaces diferentes. Diagnosticar un acceso dirigido al ambiente incorrecto mediante resolución DNS interna. Diferenciar nombre corto, nombre con namespace y FQDN de un Service. Corregir únicamente la referencia DNS responsable y validar el acceso al ambiente esperado.

Requisitos previos

- Haber completado la Práctica 4 y conservar los namespaces development y production.
- Conservar los Deployments webapp y Services webapp-svc creados en ambos namespaces.
- Tener Docker Desktop, Minikube y kubectl operativos.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Disponer del archivo dns-client4-1.yaml utilizado para generar el escenario de diagnóstico.

Duración estimada

20 minutos

Introducción

En este reto analizarás un cliente ejecutándose dentro de development que fue configurado para consultar la aplicación de production, pero obtiene la dirección de otro Service. No conocerás inicialmente la causa. Deberás comparar recursos, namespaces, ClusterIP, configuración DNS y resultados de resolución para determinar qué nombre está resolviendo Kubernetes, explicar por qué ocurre y corregir únicamente la referencia utilizada por el cliente.

Instrucciones



Escenario del reto

Los namespaces `development` y `production` contienen recursos con nombres equivalentes:

```
Deployment: webapp
Service:    webapp-svc
```

 Copiar

Un cliente de diagnóstico ubicado en `development` debía consultar la aplicación desplegada en `production`. Sin embargo, al resolver el destino configurado obtiene una dirección que no corresponde al Service esperado.

Durante el reto deberás responder:

1. ¿Qué Service está resolviendo realmente el cliente?
2. ¿Por qué Kubernetes devuelve esa dirección?
3. ¿Qué papel tiene el namespace del Pod en la resolución DNS?
4. ¿Qué nombre permite identificar de forma explícita al Service de production?
5. ¿Cómo puede corregirse el cliente sin modificar los Services existentes?

 Copiar

Distribución sugerida

Preparación	2 min
Reto 1	6 min
Reto 2	7 min
Reto 3	5 min
Total	20 min

 Copiar

Preparación del escenario

Confirmar los ambientes reutilizados

- **Paso 1.** Comprueba que los namespaces `development` y `production` siguen activos antes de iniciar el diagnóstico, porque el escenario depende de recursos creados en la práctica principal.

```
kubectl get namespaces development production
```

 Copiar

- **Paso 2.** Consulta los Deployments y Services de `development` para confirmar que la aplicación local continúa disponible y que no existe un fallo previo en ese ambiente.

```
kubectl get deployment,service -n development
```

 Copiar

- **Paso 3.** Consulta los mismos tipos de recursos en `production` para verificar que el Service remoto también existe antes de evaluar la resolución DNS del cliente.

```
kubectl get deployment,service -n production
```

 Copiar

Registrar las direcciones internas

- **Paso 4.** Obtén la ClusterIP de `webapp-svc` en `development`; esta dirección servirá como referencia para identificar posteriormente qué ambiente está resolviendo el cliente.

```
echo "Service development:"  
kubectl get service webapp-svc \  
-n development \  
-o jsonpath='{.spec.clusterIP}{"\n"}'
```

 Copiar

- **Paso 5.** Obtén la ClusterIP del Service con el mismo nombre en `production` para disponer de una segunda referencia y comparar ambas respuestas DNS.

```
echo "Service production:"  
kubectl get service webapp-svc \  
-n production \  
-o jsonpath='{.spec.clusterIP}{"\n"}'
```

 Copiar

IMPORTANTE: Las dos ClusterIP deben ser diferentes. Los Services pueden compartir nombre porque pertenecen a namespaces distintos, pero cada recurso posee identidad y dirección propias.

Descargar y aplicar el escenario

- **Paso 6.** Descarga el manifiesto del cliente de diagnóstico desde el repositorio del curso; este archivo contiene la configuración que originó el comportamiento que deberás investigar.

```
curl -L \  
  -o dns-client4-1.yaml \  
  https://raw.githubusercontent.com/Netec-  
Mx/CKA/refs/heads/main/labs/lab7/dns-client4-  
1.yaml
```

[Copiar](#)

IMPORTANTE: No abras ni modifiques `dns-client4-1.yaml` antes de completar los primeros dos retos. El objetivo es diagnosticar el problema utilizando el estado efectivo del recurso.

- **Paso 7.** Aplica el manifiesto para crear el Pod `environment-client` dentro de `development`, reproduciendo el estado defectuoso que analizarás durante la práctica.

```
kubectl apply -f dns-client4-1.yaml
```

[Copiar](#)

- **Paso 8.** Espera a que el Pod alcance condición `Ready`, evitando interpretar errores de inicio del contenedor como parte del problema DNS planteado en el reto.

```
kubectl wait \  
  --for=condition=Ready \  
  pod/environment-client \  
  -n development \  
  --timeout=60s
```

[Copiar](#)

Reto 1. Determinar qué ambiente está resolviendo el cliente

Tiempo sugerido: 6 minutos

En este reto debes demostrar primero qué recurso está alcanzando realmente el cliente antes de intentar corregir cualquier configuración.

Reto 1.1. Confirmar la ubicación del cliente

- **Paso 9.** Consulta el Pod con salida extendida para confirmar que `environment-client` está ejecutándose específicamente dentro del namespace `development`.

```
kubectl get pod environment-client \
  -n development \
  -o wide
```

[Copiar](#)

Reto 1.2. Consultar el destino configurado

- **Paso 10.** Extrae el valor efectivo de la variable `TARGET_SERVICE` directamente desde la especificación del Pod para conocer el nombre que intenta resolver la aplicación cliente.

```
kubectl get pod environment-client \
  -n development \
  -o jsonpath='{.spec.containers[0].env[?(@.name=="TARGET_SERVICE")].value}'
```

[Copiar](#)

Reto 1.3. Resolver el nombre desde el Pod

- **Paso 11.** Ejecuta `nslookup` desde el propio cliente utilizando la variable configurada, de forma que la prueba utilice exactamente el mismo nombre que recibe el contenedor.

```
kubectl exec environment-client \
  -n development -- \
  sh -c 'nslookup "$TARGET_SERVICE"'
```

[Copiar](#)

- **Paso 12.** Compara la dirección devuelta por DNS con las dos ClusterIP registradas durante la preparación para identificar a qué ambiente pertenece realmente la respuesta.

Evidencias que debes obtener

1. ¿Cuál es el valor actual de TARGET_SERVICE?
2. ¿Qué dirección IP devuelve nslookup?
3. ¿Esa IP pertenece a development o production?
4. ¿El Pod cliente está consultando el ambiente esperado?
5. ¿El problema parece estar en la aplicación, en el Service o en la referencia DNS?

 Copiar

Resultado esperado:

Debes poder identificar qué Service está resolviendo el nombre configurado sin modificar todavía el Pod, los Services ni CoreDNS.

Reto finalizado

Resultado esperado: Namespaces `development` y `production` comparados correctamente, identificando Services con el mismo nombre pero ClusterIP independientes.



Reto 2. Explicar la causa mediante DNS y namespaces

Tiempo sugerido: 7 minutos

Ahora debes explicar por qué Kubernetes resolvió ese Service y demostrar cómo cambia el resultado cuando el nombre incluye el namespace del destino.

Reto 2.1. Inspeccionar la configuración DNS del Pod

- **Paso 13.** Consulta `/etc/resolv.conf` dentro del cliente para identificar el servidor DNS y los dominios de búsqueda que Kubernetes configuró para un Pod ubicado en `development`.

```
kubectl exec environment-client \  
-n development -- \  
sh -c 'cat /etc/resolv.conf'
```

 Copiar

Localiza especialmente entradas similares a:

```
nameserver ...  
search development.svc.cluster.local  
svc.cluster.local cluster.local  
options ndots:5
```

 Copiar

NOTA: Los valores exactos pueden variar según el clúster, pero el dominio de búsqueda del namespace del Pod debe permitir reconocer cómo se expande un nombre corto.

Reto 2.2. Comparar tres formas de resolución

- **Paso 14.** Resuelve únicamente el nombre corto para observar cómo Kubernetes utiliza el namespace local del Pod como contexto de búsqueda.

```
kubectl exec environment-client \  
-n development -- \  
nslookup webapp-  
svc.production.svc.cluster.local
```

 Copiar

- **Paso 15.** Resuelve el nombre incluyendo explícitamente `production` para comprobar que el namespace forma parte de la identidad DNS del Service remoto.

```
kubectl exec environment-client \  
-n development -- \  
nslookup webapp-  
svc.production.svc.cluster.local
```

 Copiar

Reto 2.3. Relacionar resultados

Compara las direcciones obtenidas con:

```
kubectl get service webapp-svc \
  -n development \
  -o jsonpath='{.spec.clusterIP}{"\n"}'

kubectl get service webapp-svc \
  -n production \
  -o jsonpath='{.spec.clusterIP}{"\n"}'
```

 Copiar

Preguntas de análisis

1. ¿Qué nombre resuelve al Service de development?
2. ¿Qué nombres identifican correctamente al Service de production?
3. ¿Qué función cumple development.svc.cluster.local dentro de search?
4. ¿Por qué dos Services pueden compartir el nombre webapp-svc?
5. ¿Un namespace bloquea automáticamente el tráfico hacia otro namespace?
6. ¿Qué nombre ofrece la referencia más explícita al Service remoto?

 Copiar

IMPORTANTE: Los namespaces proporcionan separación lógica y ámbito de nombres, pero no constituyen por sí mismos una barrera de red. El control de tráfico requiere mecanismos como NetworkPolicy cuando el CNI utilizado las implementa.

Diagnóstico requerido

Antes de continuar debes poder completar correctamente esta explicación:

El Pod environment-client está en el namespace

 Copiar

_____.

TARGET_SERVICE contiene el nombre

_____.

Al utilizar un nombre corto, Kubernetes lo resuelve dentro del contexto

DNS del namespace _____.

Por eso el cliente obtiene la ClusterIP perteneciente a _____.

Para identificar explícitamente el Service remoto debe incluirse

_____ en el nombre DNS.

Reto finalizado

Resultado esperado: Comportamiento DNS diagnosticado correctamente, comprobando que el nombre corto resuelve al Service local y que el FQDN identifica explícitamente `production`.

Reto 3. Corregir únicamente la referencia del cliente

Tiempo sugerido: 5 minutos

Ya identificaste la causa. La corrección debe limitarse al cliente que utiliza la referencia equivocada.

Restricciones del reto

- [] No modifiques el Deployment webapp de development.
- [] No modifiques el Deployment webapp de production.
- [] No modifiques ninguno de los Services webapp-svc.
- [] No cambies CoreDNS.
- [] No muevas environment-client a otro namespace.

 Copiar

Reto 3.1. Corregir el manifiesto

- **Paso 16.** Abre `dns-client4-1.yaml` y modifica únicamente el valor de `TARGET_SERVICE` para que identifique explícitamente `webapp-svc` dentro de `production`.

Puedes utilizar una de estas referencias:

```
webapp-svc.production
webapp-svc.production.svc.cluster.local
```

 Copiar

RECOMENDACIÓN: Utiliza el FQDN completo si deseas que el manifiesto deje explícito el namespace y el dominio DNS del Service objetivo.

Reto 3.2. Recrear únicamente el cliente

- **Paso 17.** Elimina el Pod existente porque las variables de entorno definidas en la especificación de un Pod no se actualizan modificando el contenedor que ya está ejecutándose.

```
kubectl delete pod environment-client \
-n development
```

 Copiar

- **Paso 18.** Aplica nuevamente el manifiesto corregido para crear el cliente con el nuevo valor de `TARGET_SERVICE`.

```
kubectl apply -f dns-client4-1.yaml
```

 Copiar

- **Paso 19.** Espera a que el nuevo Pod esté listo antes de ejecutar las comprobaciones finales de DNS y conectividad.

```
kubectl wait \  
  --for=condition=Ready \  
  pod/environment-client \  
  -n development \  
  --timeout=60s
```

 Copiar

Reto 3.3. Validar el destino corregido

- **Paso 20.** Comprueba el valor final de `TARGET_SERVICE` para confirmar que el nuevo Pod utiliza la referencia remota esperada.

```
kubectl get pod environment-client \  
  -n development \  
  -o jsonpath='{.spec.containers[0].env[? (  
  (@.name=="TARGET_SERVICE")).value]}'
```

 Copiar

- **Paso 21.** Resuelve nuevamente la variable desde el contenedor y confirma que la dirección ahora coincide con la ClusterIP de `production`.

```
kubectl exec environment-client \  
  -n development -- \  
  sh -c 'nslookup "$TARGET_SERVICE"'
```

 Copiar

- **Paso 22.** Realiza una solicitud HTTP al destino configurado para validar que la corrección no solo resuelve DNS, sino que también permite alcanzar el Service remoto.

```
kubectl exec environment-client \  
  -n development -- \  
  sh -c 'wget -qO- "http://$TARGET_SERVICE" |  
  head -5'
```

 Copiar

Reto 3.4. Retirar el recurso temporal

- **Paso 23.** Elimina únicamente `environment-client` después de conservar la evidencia, manteniendo intactos los recursos de ambos ambientes para prácticas posteriores.

```
kubectl delete pod environment-client \
-n development
```

[Copiar](#)

Reto finalizado

Resultado esperado: Referencia DNS del cliente corregida sin modificar Services ni CoreDNS, validando resolución hacia `production`, conectividad HTTP y eliminación del Pod temporal.

✓ Validación final del reto

La práctica se considera completada cuando puedes demostrar que el problema estaba relacionado con la forma de identificar el Service remoto y no con una falla de CoreDNS o de los workloads.

Criterios de éxito

```
[✓] development y production continúan Active.
[✓] webapp permanece disponible en ambos namespaces.
[✓] webapp-svc conserva una ClusterIP diferente en
cada ambiente.
[✓] El nombre corto webapp-svc desde development
resuelve al Service local.
[✓] El nombre con namespace o FQDN resuelve al
Service de production.
[✓] TARGET_SERVICE fue corregido sin modificar los
Services.
[✓] La petición HTTP hacia production funciona desde
development.
[✓] environment-client fue eliminado al finalizar.
```

[Copiar](#)

- **Paso 24.** Comprueba los recursos finales de ambos ambientes para confirmar que ningún Deployment o Service fue alterado durante el

troubleshooting.

```
kubectl get deployment,service -n development  
kubectl get deployment,service -n production
```

[Copiar](#)

- **Paso 25.** Genera una vista global de los Services `webapp-svc` para observar que el namespace distingue recursos con nombres equivalentes.

```
kubectl get services --all-namespaces | grep  
webapp-svc
```

[Copiar](#)

- **Paso 26.** Confirma que el Pod temporal ya no permanece en `development` después de terminar la validación.

```
kubectl get pod environment-client \  
-n development \  
--ignore-not-found
```

[Copiar](#)

IMPORTANTE: Conserva los namespaces `development` y `production` junto con sus aplicaciones. No elimines el clúster Minikube al terminar esta práctica.

🔧 Resolución de problemas

Problema 1. environment-client no alcanza estado Ready

Síntoma: El Pod permanece en `Pending`, `ContainerCreating` o `ImagePullBackOff`.

Análisis: Este problema ocurre antes de ejecutar cualquier consulta DNS, por lo que primero debes comprobar que el contenedor pudo programarse e iniciar correctamente.

```
kubectl get pod environment-client \  
-n development  
  
kubectl describe pod environment-client \  
-n development
```

[Copiar](#)

Revisa especialmente los `Events` para identificar problemas de scheduling o descarga de la imagen.

Problema 2. webapp-svc no resuelve mediante nombre corto

Síntoma: `nslookup webapp-svc` falla desde `environment-client`.

Análisis: El escenario supone que el Service local existe. Verifica primero ese recurso antes de concluir que CoreDNS presenta una falla.

```
kubectl get service webapp-svc \
-n development

kubectl get endpoints webapp-svc \
-n development
```

[Copiar](#)

Si el Service existe, confirma que CoreDNS se encuentre operativo:

```
kubectl get pods \
-n kube-system \
-l k8s-app=kube-dns
```

[Copiar](#)

Problema 3. El nombre de production no devuelve la ClusterIP esperada

Síntoma: La dirección obtenida no coincide con la registrada para `webapp-svc` en `production`.

Análisis: Comprueba primero cuál referencia estás resolviendo y compara directamente contra la ClusterIP efectiva del Service remoto.

```
kubectl get service webapp-svc \
-n production \
-o jsonpath='{.spec.clusterIP}'

kubectl exec environment-client \
-n development -- \
nslookup webapp-svc.production.svc.cluster.local
```

[Copiar](#)

Problema 4. DNS resuelve correctamente pero la petición HTTP falla

Síntoma: `nslookup` devuelve la ClusterIP de `production`, pero `wget` no obtiene respuesta.

Análisis: En este punto DNS ya funciona. Debes revisar si el Service posee backends disponibles y si los Pods de la aplicación están preparados para recibir tráfico.

```
kubectl get endpoints webapp-svc \
-n production

kubectl get pods \
-n production \
-l app=webapp
```

[Copiar](#)

Si no aparecen endpoints, investiga la relación entre selector del Service y labels de los Pods antes de modificar DNS.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar el reto habrás diagnosticado una referencia DNS incorrecta entre namespaces, demostrado cómo Kubernetes resuelve nombres cortos dentro del namespace local y restaurado el acceso explícito hacia el Service `webapp-svc` del ambiente `production` sin modificar los recursos de aplicación.

Notas y consideraciones

- Esta práctica dura 20 minutos y reutiliza los recursos creados en el Lab 4.
- No elimines ni modifiques los Deployments o Services de `development` y `production`.
- El escenario está diseñado para resolverse desde el cliente de diagnóstico; no modifiques CoreDNS.
- Conserva los namespaces y aplicaciones después de completar la práctica.
- Esta práctica complementaria no incluye prompts de apoyo con IA.

URL de referencia

- [Namespaces](#)
- [DNS for Services and Pods](#)

- [Services](#)